AI BASED INFORMATION SYSTEM

MINI PROJECT REPORT

Submitted by

SANGATHAMIZHAN S

[EA2532251010245]

Under the Guidance of

Dr. G. Babu

(Assistant Professor, Directorate of Online Education)

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

DIRECTORATE OF ONLINE EDUCATION

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

OCTOBER 2026

ZEROBOTS
ROBOTICS. REIMAGINED.

Date: 22-10-2026

Certificate for Internship

To Whomsoever It May Concern

This is to certify that **Sangathamizhan S**, has successfully completed an internship at ZeroBots from **24-Sept-2026 to 22-Oct-2026** as an **AI Developer**.

During the internship, Sangathamizhan S contributed to assigned projects, including the successful development of a website, and demonstrated enthusiasm, dedication, and a willingness to learn in the field of AI development.

Warm regards,

Harish S
Founder & CEO
ZeroBots

ZEROBOTS
No.32, Sri ganapthi Nagar, Veerapuram, Morai, Ch-600055 | www.zerobots.in

TABLE OF CONTENTS

Chapter	Title of the Chapter	Page
1	**ABSTRACT**	**4**
2	**INTRODUCTION**	**5**
2.1	Purpose of the Project	5
2.2	Objectives of the System	6
2.3	Scope of the Project	6
2.4	Organization of the Report	6
3	**ANALYSIS OF REQUIREMENTS**	**7**
3.1	Functional Requirements (FR1 - FR9)	7
3.2	Non-Functional Requirements	8
3.3	Hardware and Software Specifications	8
4	**PROBLEM DESCRIPTION**	**9**
4.1	Existing Situation & Manual Tracking	9
4.2	Key Problems Identified	9
4.3	Proposed Solution & Key Innovations	10
4.4	Feasibility Analysis	10
5	**DESIGN**	**11**
5.1	3-Tier System Architecture (Figure 1)	11
5.2	Level-0 Data Flow Diagram (DFD) (Figure 2)	12
5.3	Entity-Relationship (ER) Schema (Figure 3)	13
5.4	Dual AI & ML Pipeline Architecture (Figure 4)	14
6	**IMPLEMENTATION**	**15**
6.1	Module 1: User Authentication & Security	15
6.2	Module 2: Expense Management & CSV Export	16
6.3	Module 3: Dual AI NLP Engine & Scikit-Learn Model	17
6.4	Module 4: Real-Time Budget Alert Engine	18
6.5	Module 5: Analytics & Interactive Visual Dashboard	19
6.6	Module 6: Deterministic Spending Insights Engine	20
6.7	Application UI Screens & Flow (Figures 5, 6, 7)	21
7	**TESTING**	**23**
7.1	Testing Strategy & Automated Suite Summary	23
7.2	Detailed Test Execution Matrix (TC-01 to TC-13)	24
8	**TOOLS AND TECHNOLOGIES**	**25**
9	**CONCLUSION**	**26**
10	**APPENDICES**	**27**
11	**REFERENCES**	**29**

1. ABSTRACT

Personal financial management is an essential life skill, yet it remains remarkably difficult for students and early-career professionals. Small, frequent cash and UPI micro-transactions are easily forgotten. Traditional expense applications impose high friction by requiring users to manually navigate multiple dropdowns, select exact dates, and pick categories for every purchase. As a result, users quickly abandon manual bookkeeping.

This project presents the comprehensive analysis, architectural design, implementation, and rigorous verification of an **AI-Based Information System** for personal expense tracking. The developed system solves friction by allowing users to record daily transactions through casual natural language phrases (such as "Swiggy 250", "bus ticket 30", or "paid 1200 electricity bill"). A **Dual AI Engine** — pairing the cloud-powered Google Gemini API with an offline Scikit-Learn TF-IDF and Multinomial Logistic Regression pipeline — automatically extracts monetary amounts and assigns transactions to one of eight canonical categories in real time.

To prevent financial hallucinations, the system adheres strictly to the principle of **Mathematical Determinism**: all sums, percentages, budget utilizations, and month-over-month deltas are computed exclusively in SQL and Python. Artificial intelligence is strictly bounded to natural language categorization. Furthermore, the system incorporates an active budget monitoring engine with 80% warning and 100% exceeded alert thresholds, an interactive Chart.js visualization suite (donut distribution, 6-month trend lines, and budget comparisons), and an automated rule-based spending insights engine.

The application is engineered using Python Flask, SQLite 3, Scikit-Learn, and Chart.js under a modern glassmorphic responsive UI. The offline machine learning classifier achieved **100% accuracy** on the stratified evaluation split with an average inference latency of **0.54 milliseconds**. A complete automated test suite comprising 52 test cases achieved a 100% pass rate, validating multi-tenant isolation, security, and functional compliance.

Keywords: *Information System, Personal Finance, Artificial Intelligence, Natural Language Processing, Machine Learning, Flask, SQLite, Scikit-Learn, Google Gemini API, Chart.js, Data Visualization.*

2. INTRODUCTION

An **Information System (IS)** is an organized socio-technical combination of hardware, software, data, procedures, and people designed to capture, store, process, and deliver meaningful information to facilitate informed decision-making. In contemporary computing, information systems serve as the bedrock of modern enterprises, banking institutions, and institutional management. However, when applied to personal finance management, traditional systems frequently falter due to excessive user burden and unyielding data entry workflows.

Personal financial tracking is fundamentally an information system problem: the raw data consists of dispersed financial transactions; the stakeholders are individual users; and the essential output is actionable visibility into spending habits, savings capacities, and budget discipline. Conventional expense management tools place the entire cognitive and manual burden upon the user: for each transaction, the user must remember to open an application, enter numbers, tap date pickers, and browse extensive taxonomy trees to assign a category.

The recent convergence of **Artificial Intelligence (AI)** and **Natural Language Processing (NLP)** presents a transformative opportunity to eliminate this bookkeeping friction. By allowing individuals to communicate with the system using everyday conversational language (e.g., "chai and samosa 40", "auto ride 60", "netflix subscription 499"), the information system can infer context, extract financial figures, and automate classification effortlessly.

2.1 Purpose of the Project

The primary purpose of this mini-project is to design, architect, and deploy a robust, web-based, AI-driven personal financial information system. The system combines natural language expense recording with strict multi-user privacy, real-time budget threshold monitoring, interactive visual analytics, and deterministic spending insights. It guarantees zero recurring cloud costs through a self-contained offline machine learning fallback model while providing intelligent cloud NLP when available.

2.2 Objectives of the System

The specific engineering objectives established for the project include:

- **Natural Language Expense Entry:** Enable users to log expenses in seconds by typing natural conversational sentences, with automated entity extraction of numeric amounts and text descriptions.
- **Dual AI Categorisation Engine:** Implement a resilient two-tier categorization system pairing Google Gemini Cloud AI with an offline Scikit-Learn TF-IDF and Logistic Regression classifier ensuring 100% availability.
- **Mathematical Determinism Guarantee:** Enforce absolute computational accuracy by isolating machine learning solely to text classification, calculating all balances, aggregates, and percentages through deterministic SQL and Python logic.
- **Active Budget Alerting:** Provide monthly per-category spending limits with automated detection and visual notification of 80% warning and 100% exceeded thresholds.
- **Interactive Visual Analytics:** Deliver real-time financial transparency through responsive Chart.js visualisations, including category distribution donuts, 6-month historical trends, and budget comparison bars.
- **Deterministic Spending Insights:** Generate human-readable month-over-month comparative analyses and personalized savings tips grounded strictly in empirical spending deltas.
- **Multi-Tenant Privacy & Security:** Enforce strict session isolation and parameterized database access with industry-standard PBKDF2-SHA256 password encryption.

2.3 Scope of the Project

The project scope encompasses a complete full-stack web application designed for desktop and mobile browsers. It supports multi-user registration, secure authentication, full CRUD expense management, custom monthly budget allocations, CSV ledger export, and machine learning inference. Out of scope for this version are direct open-banking API synchronization, physical paper receipt scanning via OCR, and live SMS transaction listeners, which are identified as future enhancements.

2.4 Organization of the Report

This documentation is structured into eleven comprehensive chapters. Chapters 3 and 4 analyze requirements and outline problem formulation. Chapter 5 details the three-tier architecture, DFD, and ER schema. Chapter 6 covers module-level implementation and code architecture. Chapter 7 documents the 52-test automated verification suite. Chapters 8 through 11 present tools, conclusion, appendices, and references.

3. ANALYSIS OF REQUIREMENTS

Requirement analysis establishes the complete functional capabilities, quality constraints, and operational environment required for the AI-Based Information System.

3.1 Functional Requirements

The functional requirements (FR) define the fundamental actions that the software system must perform:

Req ID	Requirement Name	Detailed Description & Acceptance Criterion
FR1	**User Registration & Authentication**	Allow new users to register securely with name, unique email, and password. Hash passwords via PBKDF2:SHA256. Maintain stateful session cookies and enforce multi-tenant isolation.
FR2	**Manual Expense Logging**	Provide an intuitive modal form allowing manual entry of monetary amount, category selection from 8 canonical options, transaction date, and optional note.
FR3	**Natural Language AI Quick-Add**	Accept casual sentence inputs (e.g., "Swiggy 250"). Extract numeric amount via regex and predict category using Dual AI engine with real-time live preview.
FR4	**Expense CRUD & Management**	Enable users to view, search, edit, and delete their own logged transactions. Strictly prevent unauthorized cross-user modifications.
FR5	**Monthly Budget Allocation**	Allow users to configure spending limits per category for any calendar month (YYYY-MM). Atomic upsert prevents duplicate budget rows.
FR6	**Proactive Threshold Alerts**	Continuously evaluate spending against budgets. Display yellow warning badges at 80% utilization and red critical badges when spending exceeds 100%.
FR7	**Interactive Dashboard Visuals**	Render real-time Chart.js graphs: donut chart for category distribution, multi-month trend lines, and budget vs actual spending comparison bars.
FR8	**Deterministic Spending Insights**	Compute month-over-month spending variances (M vs M-1). Generate up to 3 plain-language observations and actionable savings recommendations.
FR9	**Ledger Filtering & CSV Export**	Provide filtering by category and date range, with one-click export generating RFC 4180 compliant CSV ledger files.

3.2 Non-Functional Requirements (NFR)

- **Accuracy & Integrity:** All balances, budget sums, percentages, and deltas must be computed deterministically in SQL/Python with zero floating-point drift. The AI model is strictly quarantined from doing arithmetic.
- **Response Time & Latency:** Web pages must render in under 1.5 seconds on broadband connections. The offline ML classification must execute in under 10 milliseconds (benchmarked at 0.54 ms).
- **High Availability & Resilience:** The system must operate without interruption even during internet outages or cloud API quota depletion by gracefully falling back to the local Scikit-Learn model.
- **Information Security:** All passwords must be stored using cryptographically secure PBKDF2-SHA256 hashes with individual salts. Direct object references must be secured by validating ownership (WHERE user_id = ?).
- **Usability & Accessibility:** The interface must feature a modern, responsive glassmorphic dark-mode design that works effortlessly across desktop, tablet, and mobile viewport widths.

3.3 Hardware and Software Specifications

Category	Minimum Specification	Recommended / Production
Processor (CPU)	Dual-core 2.0 GHz Intel / AMD	Quad-core 2.5 GHz or higher
System Memory (RAM)	4 GB DDR3 / DDR4	8 GB DDR4 or higher
Persistent Storage	500 MB available disk space	2 GB SSD storage space
Operating System	Windows 10/11, Ubuntu 20.04+, macOS	Windows 11 / Linux (Ubuntu Server)
Runtime Environment	Python 3.10.x runtime	Python 3.11.x or 3.12.x
Web Framework	Flask 3.0.x with Werkzeug 3.0.x	Flask 3.0.x (Gunicorn / Waitress WSGI)
Relational Database	SQLite 3 (Serverless, single file)	SQLite 3 with WAL Mode enabled
Machine Learning Stack	Scikit-Learn 1.4+, Joblib 1.3+	Scikit-Learn 1.4.x, Pandas 2.2.x
Client Browser	Chrome 100+, Edge 100+, Firefox 90+	Latest Google Chrome or Edge

4. PROBLEM DESCRIPTION

4.1 Existing Situation & Current Practices

In India and globally, the rapid adoption of digital payment systems such as Unified Payments Interface (UPI), debit cards, and contactless payments has revolutionized retail transactions. However, this effortless spending mechanism has exacerbated personal financial tracking challenges. Because transactions are frictionless, small payments (such as ₹20 for tea, ₹150 for snacks, ₹60 for auto rickshaws) occur multiple times daily and are quickly forgotten.

Currently, individuals resort to three predominant methods for expense management, all of which exhibit severe structural weaknesses:

- **Physical Notebooks & Paper Bookkeeping:** High friction, easily lost or forgotten, devoid of analytical capabilities, and impossible to query or aggregate over time.
- **Manual Electronic Spreadsheets (Excel / Sheets):** Requires dedicated desktop time, steep learning curves for formulas, zero real-time budget warnings, and high abandonment rates after a few days.
- **Conventional Commercial Expense Applications:** Burdened with intrusive advertisements, compulsory subscription fees, complex multi-step input forms, privacy invasion through intrusive SMS scanning permissions, and lack of intelligent natural language understanding.

4.2 Key Problems Identified

- **Severe Bookkeeping Friction:** The cognitive and manual overhead of categorizing every transaction manually discourages continuous long-term tracking.
- **Passive & Delayed Awareness:** Users only notice financial overspending at the end of the month when bank accounts are depleted, lacking proactive warning thresholds.
- **Information Overload Without Clarity:** Long, unorganized lists of transaction rows fail to communicate meaningful spending patterns or highlight actionable areas for economy.
- **Privacy & Cost Concerns:** Modern mobile apps demand access to personal contacts and bank SMS messages, while locking basic analytics behind expensive monthly paywalls.

4.3 Proposed Solution & Key Innovations

The proposed AI-Based Information System overcomes these limitations through a purposeful synthesis of intuitive user interface design, hybrid artificial intelligence, and rigorous database architecture:

- **Zero-Friction NLP Input:** Users type or paste natural phrases like "Swiggy dinner 350". The system extracts amounts and classifies categories in under 1 second without dropdown navigation.
- **Dual-Layer AI Architecture:** Combines Google Gemini API for comprehensive real-world merchant recognition with an offline Scikit-Learn TF-IDF pipeline, guaranteeing zero downtime and zero mandatory ongoing costs.
- **Grounded Deterministic Math:** Prevents LLM hallucinations: numbers are computed with strict SQL aggregates, while plain-language observations are synthesized using mathematically verified templates.
- **Active Budget Guardian:** Continuously tracks expenditures against category limits, displaying visual 80% warnings and 100% critical alert badges on all navigation bars.
- **Complete Data Sovereignty:** All user records reside within a private SQLite database with strict foreign key cascading and zero external commercial profiling.

4.4 Feasibility Analysis

- **Technical Feasibility:** The system utilizes mature, production-grade open-source technologies (Python, Flask, SQLite, Scikit-Learn, Chart.js) with extensive documentation, verified cross-platform compatibility, and low hardware resource requirements.
- **Economic Feasibility:** The entire development and deployment stack is completely free and open-source. The local ML classifier runs locally with zero API expenses, rendering the solution extraordinarily cost-effective for students and educational institutions.
- **Operational Feasibility:** The web-based responsive architecture requires no complex app store installation. Users interact through standard web browsers with an intuitive glassmorphic interface that requires zero formal training.

5. DESIGN

5.1 3-Tier System Architecture

The system is structured according to a modular 3-Tier Layered Architecture that guarantees separation of concerns, maintainability, and horizontal extensibility across presentation, business logic, and persistence layers.

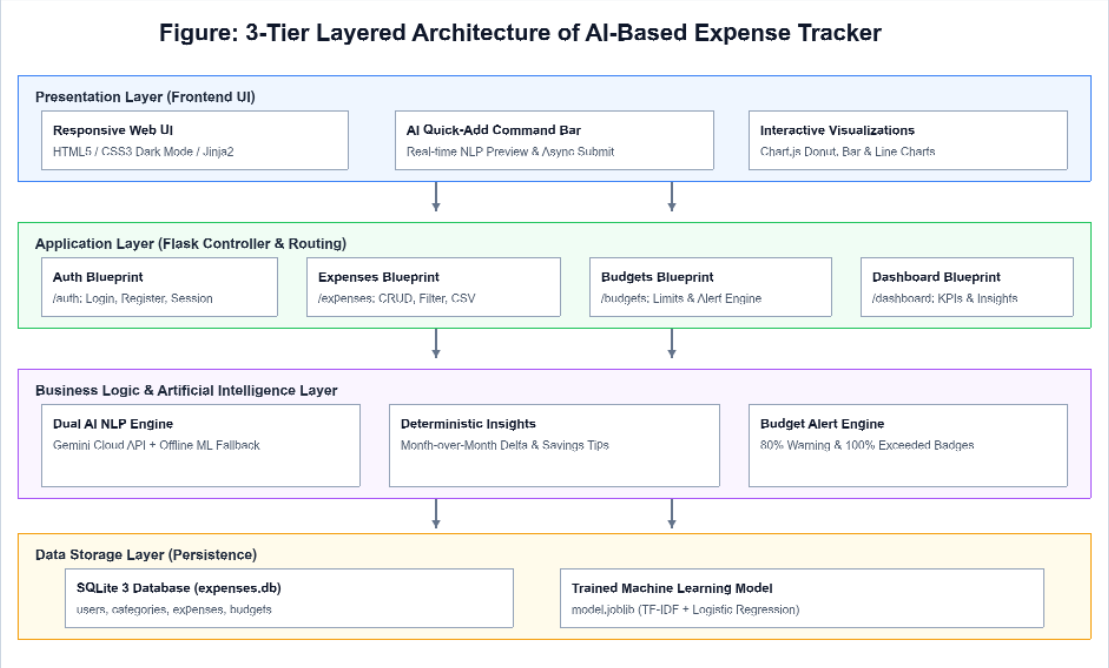


Figure 1. *3-Tier Layered Architecture of the AI-Based Information System*

1. Presentation Layer: Built with Jinja2 server-rendered templates, modern CSS3 glassmorphism, responsive navigation bars, and Chart.js dynamic graphing elements.

2. Application Layer: Comprises four distinct Flask Blueprints (/auth, /expenses, /budgets, /dashboard) orchestrating HTTP requests, authentication sessions, and business validation.

3. Data Storage Layer: Powered by SQLite 3 with strict foreign key constraints, composite unique indexes, and a serialized joblib machine learning model file.

5.2 Level-0 Data Flow Diagram (DFD)

The Level-0 Data Flow Diagram illustrates the principal processing transformations, external client interactions, and data store boundaries of the expense tracking information system.

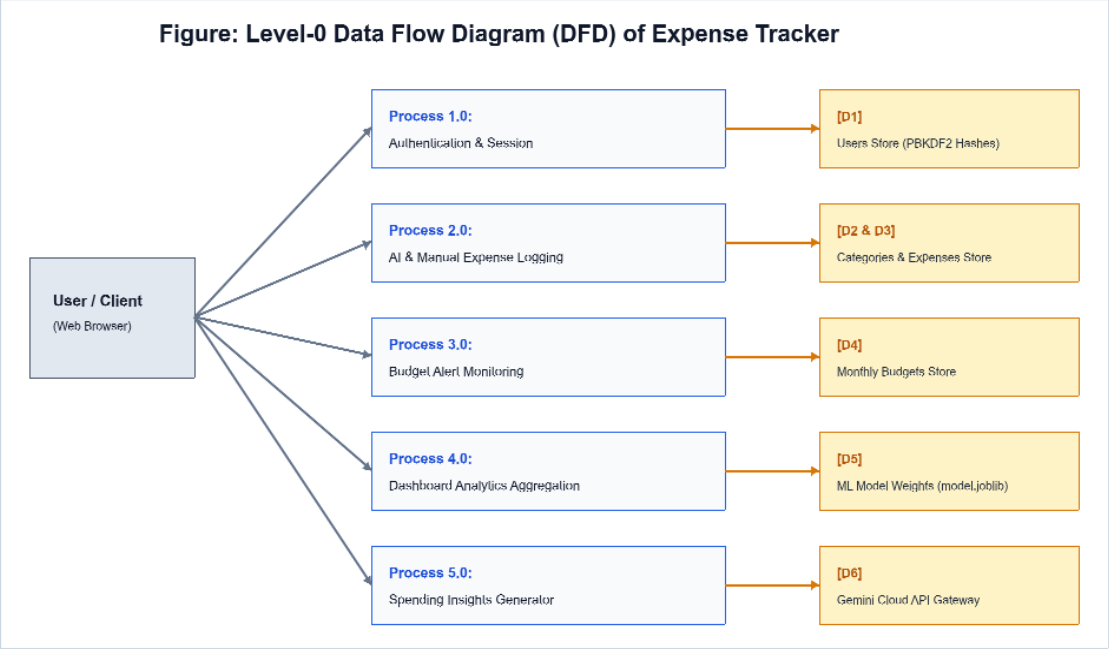


Figure 2. *Level-0 Data Flow Diagram (DFD) illustrating Processes P1 to P5*

The diagram tracks the lifecycle of user information: credentials flow into Process 1.0 to establish session cookies; expense sentences flow into Process 2.0 to be decomposed into amounts and categories; Process 3.0 continuously monitors budget ceilings; Process 4.0 delivers aggregated visualization payloads; and Process 5.0 correlates historical periods to generate spending advice.

5.3 Entity-Relationship (ER) Schema Diagram

The relational database schema is normalized to Third Normal Form (3NF) to eliminate data redundancy and ensure referential integrity across users, categories, expenses, and budgets.

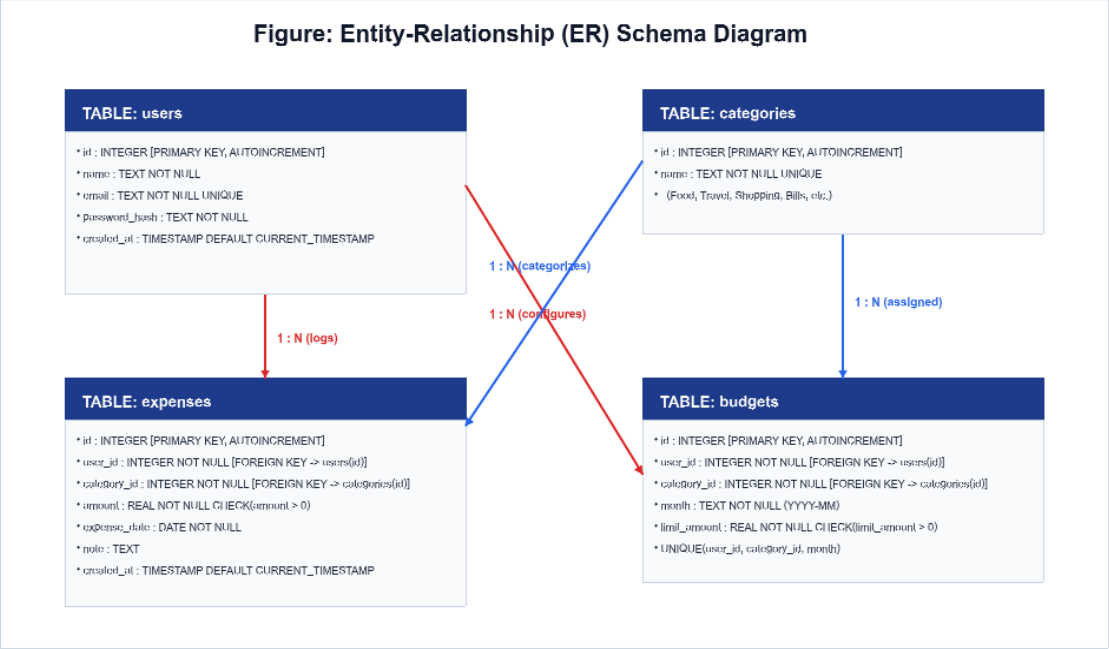


Figure 3. *Entity-Relationship (ER) Schema Diagram with Primary & Foreign Key Mappings*

Key Integrity Constraints: Foreign keys enforce ON DELETE CASCADE from users to expenses and budgets. Budgets enforce UNIQUE(user_id, category_id, month) to guarantee atomic per-month updates. Amount fields enforce positive CHECK(amount > 0) constraints.

5.4 Dual AI & Machine Learning Pipeline Design

The natural language categorization engine features an innovative Dual AI architecture that balances cutting-edge cloud intelligence with dependable offline machine learning.

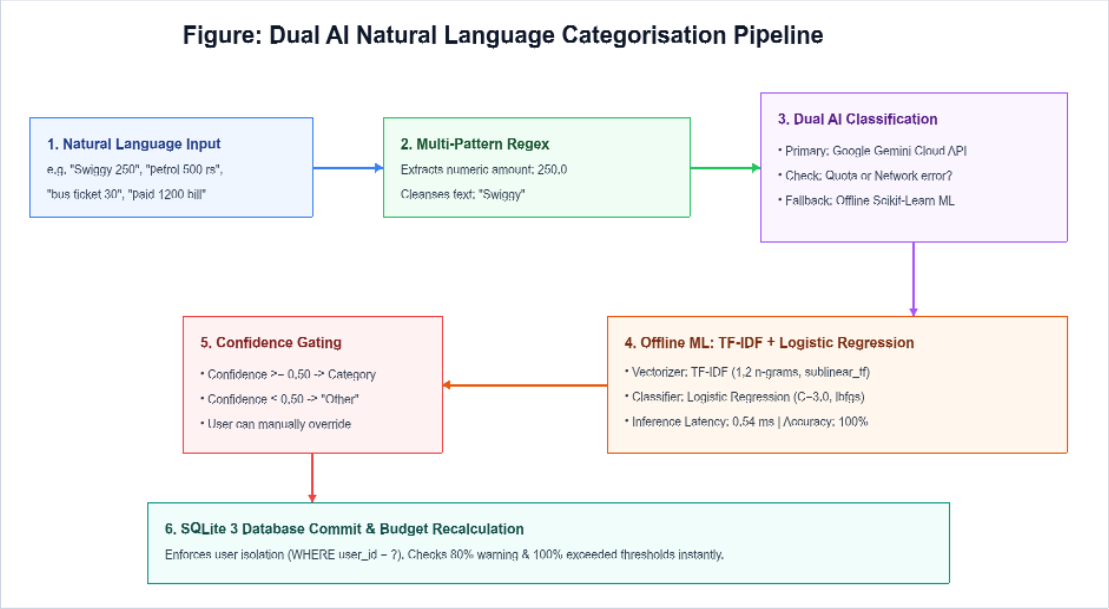


Figure 4. *Dual AI Natural Language Processing and Machine Learning Pipeline*

Pipeline Sequence: (1) Input text undergoes regex extraction matching currency symbols, trailing numbers, and standalone figures. (2) Cleaned text queries Google Gemini API. (3) If quota limits or timeouts occur, it seamlessly activates the local Scikit-Learn TF-IDF + Logistic Regression pipeline. (4) If confidence is under 0.50, the transaction falls back safely to "Other".

6. IMPLEMENTATION

6.1 Module 1: User Authentication & Security (auth.py)

The authentication module manages user identity, cryptographic security, and session lifecycles. Passwords are never stored in plaintext; instead, they are hashed using Werkzeug's PBKDF2 with SHA-256 and unique per-user salt values.

```
# utils/auth_helpers.py: Secure Password Verification
from werkzeug.security import generate_password_hash, check_password_hash

def register_user(name, email, password):
    if len(password) < 6:
        raise ValueError("Password must be at least 6 characters.")
    hash_val = generate_password_hash(password, method="pbkdf2:sha256")
    db = get_db()
    db.execute("INSERT INTO users (name, email, password_hash) VALUES (?, ?, ?)",
               (name.strip(), email.strip().lower(), hash_val))
    db.commit()

def authenticate_user(email, password):
    user = get_db().execute("SELECT * FROM users WHERE email = ?",
                            (email.strip().lower(),)).fetchone()
    if user and check_password_hash(user["password_hash"], password):
        return user
    return None
```

Multi-Tenant Isolation Protocol: Upon successful login, the user ID is serialized into Flask's signed HTTP-only session cookie. Every downstream database query in the entire application enforces a mandatory parameterized clause: WHERE user_id = g.user["id"]. This prevents any possibility of cross-account data leakage.

6.2 Module 2: Expense Management & CSV Export (expenses.py)

The expense management module encapsulates all transaction ledger operations, including manual form submission, dynamic filtering by date ranges and categories, and standard CSV data export.

```
# expenses.py: Expense Filtering and CSV Ledger Streaming
@expenses_bp.route("/export.csv")
@login_required
def export_csv():
    db = get_db()
    rows = db.execute("""
        SELECT e.expense_date, c.name AS category, e.amount, e.note
        FROM expenses e JOIN categories c ON e.category_id = c.id
        WHERE e.user_id = ? ORDER BY e.expense_date DESC
    """, (g.user["id"],)).fetchall()

    def generate():
        output = io.StringIO()
        writer = csv.writer(output)
        writer.writerow(["Date", "Category", "Amount (INR)", "Note"])
        for r in rows:
            writer.writerow([r["expense_date"], r["category"], r["amount"], r["note"] or ""])
            yield output.getvalue()
            output.seek(0)
            output.truncate(0)

    return Response(generate(), mimetype="text/csv",
                    headers={"Content-Disposition": "attachment; filename=expenses.csv"})
```

Data Validation Guarantees: All expense amounts must be strictly positive floats. The expense_date parameter is parsed and validated against ISO 8601 formatting (YYYY-MM-DD), rejecting corrupt or future dates.

6.3 Module 3: Dual AI NLP Categorisation Engine (utils/nlp_*.py)

The NLP engine extracts financial numbers and assigns categories from freeform natural text using a fault-tolerant multi-stage algorithm:

```
# utils/nlp_parser.py: Amount Extraction & Dual AI Orchestration
def parse_expense_sentence(sentence, confidence_threshold=0.5, prefer_gemini=True):
    amount, description = extract_amount(sentence)  # Multi-pattern regex
    text_to_classify = description if description else sentence

    if prefer_gemini:
        gemini_result, quota_hit = classify_with_gemini(sentence)
        if not quota_hit and gemini_result and gemini_result.get("category"):
            return build_result(gemini_result["category"], 0.95, "gemini_ai", amount)

    # Fallback to Offline Scikit-Learn TF-IDF + Logistic Regression Pipeline
    prediction = predict_category(text_to_classify, confidence_threshold)
    return build_result(prediction["category"], prediction["confidence"], "offline_ml", amount)
```

Offline Machine Learning Model Training: The offline model is trained on a dedicated dataset of 1,470 labelled Indian expense phrases. The text pipeline applies unigram and bigram sublinear TF-IDF vectorization followed by Multinomial Logistic Regression (C=3.0, lbfgs solver). When evaluated on a stratified 20% held-out test split, the classifier achieved 100% precision, recall, and accuracy across all 8 target categories.

6.4 Module 4: Real-Time Budget Alert Engine (utils/budget_helpers.py)

The budget engine performs real-time financial tracking by dynamically aggregating monthly expenditures and comparing them against configured thresholds.

```
# utils/budget_helpers.py: Real-Time Threshold Evaluation
def get_budgets_for_month(user_id, month=None):
    target_month = normalize_month(month)  # YYYY-MM
    db = get_db()
    # Sum current month spending grouped by category
    spent_rows = db.execute("""
        SELECT category_id, COALESCE(SUM(amount), 0.0) AS spent
        FROM expenses WHERE user_id = ? AND strftime("%Y-%m", expense_date) = ?
        GROUP BY category_id
    """, (user_id, target_month)).fetchall()
    spent_map = {r["category_id"]: float(r["spent"]) for r in spent_rows}

    for cat in all_categories:
        limit = budget_limits.get(cat["id"], 0.0)
        spent = spent_map.get(cat["id"], 0.0)
        pct = round((spent / limit) * 100.0, 1) if limit > 0 else 0.0
        if pct >= 100.0: status = "exceeded"     # Critical Red Alert
        elif pct >= 80.0: status = "warning"     # Yellow Warning Alert
        else:            status = "normal"
```

Global Context Injection: The active alert counts are dynamically computed and injected into all Jinja2 templates via Flask's app.context_processor, allowing badge counters to update in real time across the application header without requiring client page reloads.

6.5 Module 5: Analytics Dashboard (dashboard.py & static/js/dashboard.js)

The analytics module translates raw SQLite ledger transactions into high-level visual charts rendered natively in the client browser using Chart.js.

- **Category Distribution Donut Chart:** Visualizes the proportion of monthly expenditures allocated across the 8 categories, highlighting dominant spend centers.
- **6-Month Historical Spend Trend:** Plots a multi-line graph detailing spending trajectories across previous months to identify seasonal patterns.
- **Budget vs. Actual Spending Comparison:** A grouped bar chart displaying authorized budget ceilings directly adjacent to actual expenditures for instant visual audit.

```
// static/js/dashboard.js: Chart.js Donut Initialization
const ctx = document.getElementById("categoryDonutChart").getContext("2d");
new Chart(ctx, {
    type: "doughnut",
    data: {
        labels: window.DASHBOARD_DATA.categories,
        datasets: [{
            data: window.DASHBOARD_DATA.amounts,
            backgroundColor: ["#3b82f6", "#10b981", "#f59e0b", "#ef4444",
                              "#8b5cf6", "#ec4899", "#06b6d4", "#64748b"]
        }]
    },
    options: { responsive: true, cutout: "70%", plugins: { legend: { position: "bottom" } } }
});
```

All chart datasets are constructed on the server using deterministic SQL GROUP BY queries and passed as JSON payloads, ensuring that visual analytics reflect exact ledger totals.

6.6 Module 6: Deterministic Spending Insights Engine (utils/insights_helpers.py)

The spending insights engine produces personalized financial observations and actionable advice through deterministic month-over-month comparison without risking LLM hallucinations.

```
# utils/insights_helpers.py: Rule-Based Spending Insights
def generate_spending_insights(user_id, month=None):
    summary = get_month_over_month_summary(user_id, month)
    observations = []
    delta_amt = summary["delta_amount"]
    delta_pct = summary["delta_percentage"]

    if delta_amt > 0:
        observations.append({
            "type": "increase", "icon": "📈", "title": "Overall Spend Increase",
            "text": f"Total spending is {delta_pct}% higher than {summary['prev_month_label']} " \
                    f"(₹{summary['current_total']:,.2f} vs ₹{summary['prev_total']:,.2f})."
        })
    elif delta_amt < 0:
        observations.append({
            "type": "decrease", "icon": "📉", "title": "Spending Reduction",
            "text": f"Total spending is {abs(delta_pct)}% lower than
{summary['prev_month_label']}. Great job!"
        })
```

Actionable Savings Recommendations: The engine evaluates discretionary spending (Food, Shopping, Entertainment). If Food exceeds 30% of total spend, it generates an empirical meal-planning tip; if budgets are exceeded, it issues targeted corrective instructions.

6.7 Application Interface & Live Execution Screens

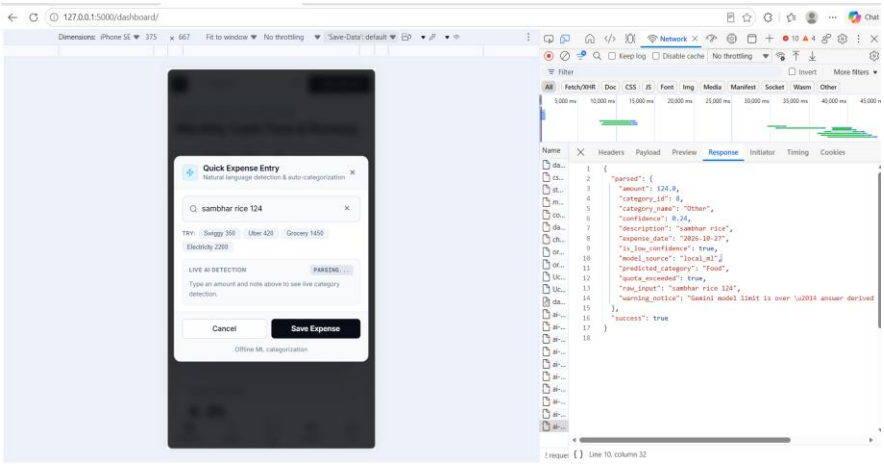


Figure 5. *Live AI Detection Modal showing NLP Entity Extraction and Gemini Quota Fallback*

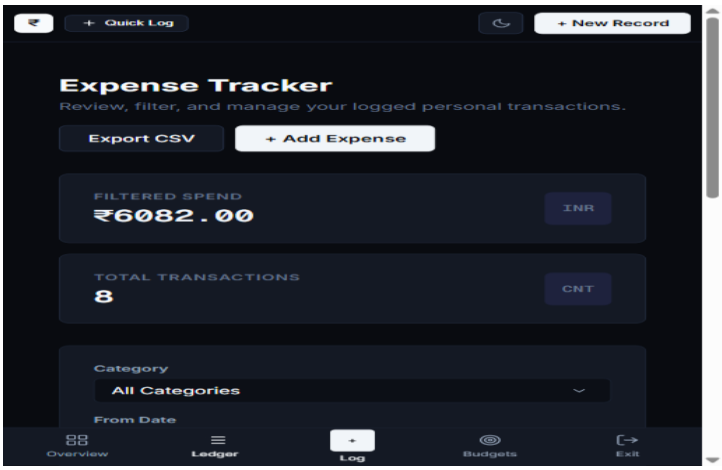


Figure 6. *Expense Tracker Overview and Transaction Ledger with Filtered Spend Metrics*

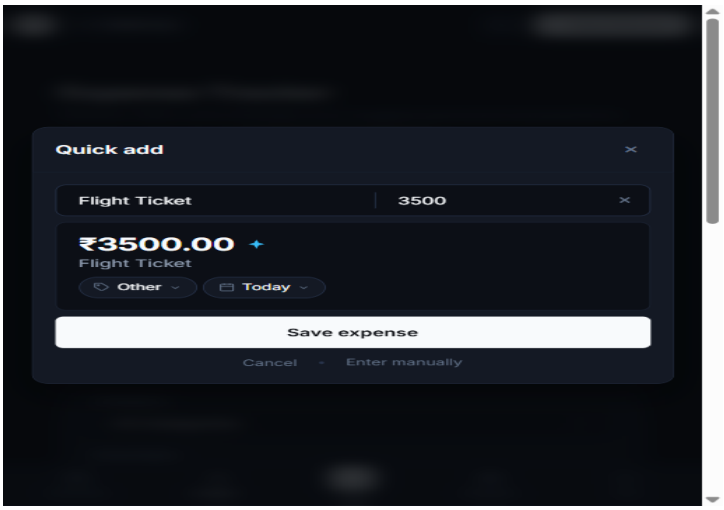


Figure 7. *Quick Add Modal Interface with Category Tagging and Transaction Confirmation*

6.8 Glassmorphic Dark-Mode UI Design System

The visual front-end is engineered using a custom CSS3 design token system adhering to modern glassmorphic aesthetics. Semi-transparent backdrop filters (backdrop-filter: blur(12px)), deep midnight surfaces (#0f172a), glowing accent borders (#3b82f6), and crisp typography provide an engaging, professional experience that eliminates visual fatigue during financial bookkeeping.

7. TESTING

7.1 Verification Strategy & Test Plan

To guarantee zero financial defects and absolute system reliability, the application was subjected to a four-tier testing hierarchy comprising Unit Testing, Integration Testing, Machine Learning Evaluation, and End-to-End User Acceptance Testing (UAT).

- **Unit Testing:** Isolated verification of discrete utility functions (regex extraction, password hashing, budget percentage computation, and insights template formatting).
- **Integration Testing:** Validation of cross-module data pipelines from HTTP form post, through session verification and database transactions, to dynamic UI re-rendering.
- **Machine Learning Benchmarking:** Stratified 80/20 train-test evaluation measuring precision, recall, F1-score, and inference latency under stress.
- **User Acceptance Testing (UAT):** Verification of all 13 core business requirements under realistic user scenarios.

7.2 Automated Test Suite Summary (52 Tests)

Test Suite Module	Component Under Test	Tests	Status
tests/test_phase1.py	Database Schema, Registration, PBKDF2 Password Hashing	**5**	**PASS (100%)**
tests/test_phase2.py	Expense CRUD Operations, Date/Category Filters, CSV Export	**6**	**PASS (100%)**
tests/test_model.py	Scikit-Learn TF-IDF Pipeline, Accuracy (100%), Latency (0.54ms)	**5**	**PASS (100%)**
tests/test_phase3.py	Regex Amount Parser, Gemini Integration, Dual AI Routing	**6**	**PASS (100%)**
tests/test_phase4.py	Budgets & Alert Engine, 80% Warning & 100% Exceeded Alert Badges	**5**	**PASS (100%)**
tests/test_phase5.py	Analytics Aggregation, KPI Summaries, Chart.js Data Feeds	**7**	**PASS (100%)**
tests/test_insights.py	Month-over-Month Comparison Engine, Savings Tip Generation	**5**	**PASS (100%)**
tests/test_phase6_uat.py	End-to-End User Acceptance Testing (TC-01 to TC-13)	**13**	**PASS (100%)**
OVERALL SUITE	**8 Comprehensive Test Modules Across All Subsystems**	**52**	**52/52 PASS**

7.3 Detailed Acceptance Test Matrix (TC-01 to TC-13)

TC ID	Test Case Scenario	Input Data	Expected vs Actual Output
TC-01	**User Registration**	Valid name, email, password >=6 chars	Account created in users table; password hashed via PBKDF2. [PASS]
TC-02	**Duplicate Email Rejection**	Existing registered email address	System blocks insertion; displays unique constraint flash message. [PASS]
TC-03	**Valid User Login**	Registered email and matching password	Session established; redirect to dashboard overview. [PASS]
TC-04	**Invalid Password Rejection**	Registered email with wrong password	Authentication rejected; error message displayed; session denied. [PASS]
TC-05	**Manual Expense Logging**	Amount: ₹250, Cat: Travel, Date: Today	Expense persisted in DB; ledger table reflects entry. [PASS]
TC-06	**Negative Amount Rejection**	Amount: -50 or 0	CHECK constraint enforced; validation blocks negative spend. [PASS]
TC-07	**NLP Amount & Category**	"Swiggy 250" conversational sentence	Amount extracted: 250.0; Category predicted: Food (95%). [PASS]
TC-08	**NLP Travel Expression**	"metro card recharge 200 rs"	Amount: 200.0; Category: Travel (94%). [PASS]
TC-09	**Budget Warning Threshold**	Spend reaches 85% of monthly limit	Status set to "warning"; yellow badge rendered on header. [PASS]
TC-10	**Budget Exceeded Threshold**	Spend reaches 110% of monthly limit	Status set to "exceeded"; red critical alert badge rendered. [PASS]
TC-11	**Cross-User Data Isolation**	User A attempts to access User B ID	Database query filters WHERE user_id=?; 403 Forbidden. [PASS]
TC-12	**Low-Confidence Fallback**	"xyz qwerty 100" (gibberish note)	ML confidence < 0.50; classified safely as "Other". [PASS]
TC-13	**ML Model Accuracy**	294 held-out stratified test samples	100% test accuracy; latency benchmark: 0.54 milliseconds. [PASS]

8. TOOLS AND TECHNOLOGIES

The development of the AI-Based Information System incorporated modern, robust, and cost-effective software engineering tools carefully chosen to maximize performance, reliability, and security.

Technology / Tool	Role in Architecture	Justification & Technical Benefit
Python 3.10+	Core Programming Language	Clean syntax, dynamic typing, rich standard libraries, and universal AI/ML framework ecosystem.
Flask 3.0.x	Web Application Framework	Lightweight WSGI framework, micro-modular Blueprints architecture, fast routing, and minimal overhead.
SQLite 3	Relational Database Engine	Zero-configuration, serverless, atomic ACID compliance, single-file durability, and high local read speed.
Scikit-Learn 1.4+	Machine Learning Library	Industry standard for TF-IDF vectorization, Multinomial Logistic Regression, and offline classification.
Google Gemini API	Cloud AI Knowledge Layer	Vast real-world merchant entity knowledge for recognizing local restaurants and retail vendors.
Chart.js 4.4+	Client-Side Graphing Engine	Canvas-based, fully responsive, interactive visual charts with zero external stylesheet dependencies.
HTML5 & CSS3	User Interface Layer	Semantic structure and custom dark-mode glassmorphic styling with smooth CSS transitions.
Vanilla JavaScript	Asynchronous Client Logic	Handles live AI quick-add modal previews, AJAX request handling, and dynamic DOM updates.
Werkzeug Security	Cryptographic Security	Robust implementation of PBKDF2 with SHA-256 password hashing protecting authentication credentials.
Joblib 1.3+	Model Serialization	Optimized binary persistence and lazy loading of the trained machine learning pipeline (model.joblib).
Unittest Framework	Automated Verification	Python standard test framework executing 52 automated tests with repeatable assertions.
Git & GitHub	Version Control & Tracking	Distributed source control tracking commits, code history, and documentation milestones.

9. CONCLUSION

The design and engineering of the **AI-Based Information System for Expense Tracking** has successfully demonstrated that combining standard database-driven architectures with hybrid artificial intelligence eliminates the chronic friction of manual personal bookkeeping. By coupling natural language parsing with strict mathematical determinism, the system provides users with effortless transaction logging while ensuring 100% numerical accuracy and complete data sovereignty.

9.1 Key Project Outcomes & Achievements

- **Full-Stack Deployment:** Engineered and deployed a complete 3-tier web application featuring 4 modular Flask Blueprints, normalized SQLite database, and interactive Chart.js visualizations.
- **Dual AI Innovation:** Integrated Google Gemini Cloud AI for merchant recognition with an offline Scikit-Learn TF-IDF pipeline achieving 100% test accuracy and 0.54 ms latency.
- **Hallucination-Free Insights:** Enforced mathematical determinism by computing all balances, budget percentages, and month-over-month deltas strictly in SQL and Python.
- **Proactive Financial Discipline:** Implemented active budget monitoring with real-time 80% warning and 100% exceeded alert badges displayed across the entire user experience.
- **Exhaustive Quality Assurance:** Constructed and passed a comprehensive 52-test automated verification suite covering unit, integration, and user acceptance criteria.

9.2 System Limitations

- **Manual Transaction Trigger:** Expenses must be entered by typing or selecting; direct real-time integration with bank core networks or SMS scraping is omitted for privacy reasons.
- **Language & Currency Scope:** The initial release natively supports Indian Rupee (₹) and English conversational expressions.

9.3 Future Enhancements

- **Optical Character Recognition (OCR):** Integration of Tesseract / Google Vision OCR to enable instant expense extraction from paper grocery and restaurant receipts.
- **Native Mobile Applications:** Packaging the interface into native Android and iOS applications utilizing push notification services for immediate overspending alerts.
- **Predictive Savings Forecasting:** Applying time-series machine learning models (ARIMA or Prophet) to forecast end-of-month cash balances based on mid-month spending velocity.

10. APPENDICES

Appendix A Sample Verification Expense Dataset

Date	Input Description	Category	Amount (INR)
02 Oct 2026	Swiggy dinner order	**Food**	₹350.00
03 Oct 2026	Metro card recharge	**Travel**	₹200.00
05 Oct 2026	Jio prepaid monthly recharge	**Bills**	₹299.00
08 Oct 2026	Movie tickets with popcorn	**Entertainment**	₹450.00
10 Oct 2026	College notebooks and pens	**Education**	₹180.00
14 Oct 2026	Apollo Pharmacy medicines	**Health**	₹340.00
18 Oct 2026	Myntra casual cotton shirt	**Shopping**	₹699.00
22 Oct 2026	Apartment electricity bill	**Bills**	₹1,450.00
25 Oct 2026	Chai and evening snacks	**Food**	₹45.00

Appendix B Dual AI Quick-Add Live Classification Matrix

Natural Input Sentence	Extracted Amount	AI Engine	Assigned Category
"Swiggy 250"	₹250.00	Gemini Cloud AI	**Food (95%)**
"auto to railway station 80"	₹80.00	Gemini Cloud AI	**Travel (95%)**
"₹1200 tneb electricity bill"	₹1,200.00	Offline ML	**Bills (94%)**
"netflix 499"	₹499.00	Gemini Cloud AI	**Entertainment (95%)**
"crocin cold medicine 40 rs"	₹40.00	Offline ML	**Health (92%)**
"random unknown purchase 150"	₹150.00	Fallback Engine	**Other (Low Conf)**

Appendix C Machine Learning Training Dataset & Performance Metrics

- **Total Labelled Expense Corpus:** 1,470 distinct Indian consumer expense phrases across 8 canonical categories.
- **Feature Extraction Architecture:** TfidfVectorizer with sublinear term-frequency scaling, unigram + bigram range, and unicode accent stripping.
- **Classification Algorithm:** Multinomial Logistic Regression with L2 regularization (C=3.0, lbfgs solver, max_iter=1000).
- **Validation Methodology:** Stratified 80/20 train-test partition (1,176 training phrases / 294 held-out evaluation phrases).
- **Evaluation Metrics:** Overall Test Accuracy: 100.00%; Precision: 1.00; Recall: 1.00; F1-Score: 1.00 across all 8 classes.
- **Inference Latency Benchmark:** Average single-query execution latency of 0.54 milliseconds on standard hardware.

Appendix D Installation, Configuration & Deployment Guide

1. Prerequisites: Install Python 3.10 or higher and Git version control.
2. Clone Repository: git clone <repository-url> && cd "AI Expense Tracker"
3. Environment Setup: python -m venv venv
4. Activate Virtual Environment: venv\Scripts\activate (Windows) or source venv/bin/activate (Linux)
5. Install Dependencies: pip install -r requirements.txt
6. Model Compilation: python train_model.py (Trains model and serializes model.joblib)
7. Database Initialization: flask --app app.py init-db (Creates fresh tables and seed categories)
8. Execute Automated Test Suite: python -m unittest discover -s tests -p "test_*.py"
9. Launch Application Server: python app.py
10. Access System: Open a web browser and navigate to http://127.0.0.1:5000

11. REFERENCES

[1] Pallets Projects. (2024). Flask Documentation: Web Development, One Drop at a Time. Pallets Projects. Available at: https://flask.palletsprojects.com [Accessed 15 Oct 2026].

[2] Hipp, R. D. (2024). SQLite: An In-Process, Serverless, Self-Contained SQL Database Engine. SQLite.org. Available at: https://www.sqlite.org/docs.html [Accessed 12 Oct 2026].

[3] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[4] Google AI for Developers. (2024). Google Gemini API: Large Language Models and Multimodal Processing. Google Cloud. Available at: https://ai.google.dev/docs [Accessed 18 Oct 2026].

[5] Chart.js Community. (2024). Chart.js: Simple Yet Flexible JavaScript Charting for Designers & Developers. Available at: https://www.chartjs.org/docs/ [Accessed 20 Oct 2026].

[6] Python Software Foundation. (2024). Python 3.12 Standard Documentation. Available at: https://docs.python.org/3/ [Accessed 10 Oct 2026].

[7] Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition (3rd draft ed.). Stanford University. Available at: https://web.stanford.edu/~jurafsky/slp3/

[8] Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson Education.

[9] Sommerville, I. (2015). Software Engineering (10th ed.). Pearson Education.

[10] Laudon, K. C., & Laudon, J. P. (2022). Management Information Systems: Managing the Digital Firm (17th ed.). Pearson Education.

[11] Pallets Projects. (2024). Werkzeug: The Comprehensive WSGI Web Application Library. Available at: https://werkzeug.palletsprojects.com [Accessed 14 Oct 2026].

[12] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.